Latihan & Praktikum
Remote Procedure Calls (RPC)
Mata Kuliah Komputasi Paralel dan Terdistribusi

Identitas Modul
Modul ini berisi latihan teori, tiga praktikum bertahap (XML-RPC, gRPC, dan toleransi kegagalan), serta satu tugas proyek mini untuk topik Remote Procedure Calls.

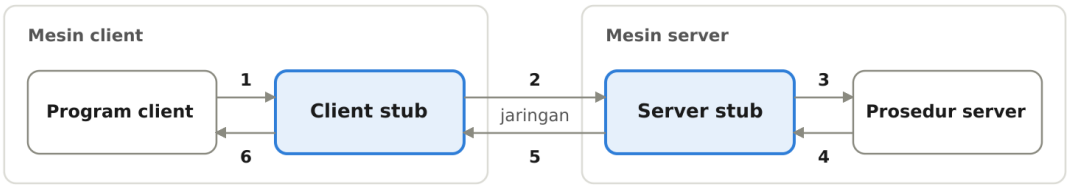
Komponen	Keterangan
Mata kuliah	Komputasi Paralel dan Terdistribusi
Topik	Remote Procedure Calls (RPC)
Alokasi waktu	2 × pertemuan praktikum (± 150 menit) + 1 minggu tugas mandiri
Bahasa	Python 3.10 ke atas
Pustaka	xmlrpc (bawaan Python), grpcio, grpcio-tools
Bentuk kerja	Praktikum individu; tugas proyek berkelompok 2–3 orang

Capaian pembelajaran
Setelah menyelesaikan modul ini, mahasiswa mampu:
1. Menjelaskan cara kerja RPC: stub, marshalling, binding, dan semantik pemanggilan.
2. Membandingkan RPC dengan pemanggilan prosedur lokal dan dengan komunikasi berbasis pesan (socket).
3. Membangun layanan client–server menggunakan XML-RPC dan gRPC.
4. Mendefinisikan antarmuka layanan dengan IDL (Protocol Buffers).
5. Menangani concurrency, timeout, dan kegagalan jaringan pada sistem RPC.
6. Mengukur kinerja (latensi dan throughput) layanan RPC.

Persiapan lingkungan
```
python3 --version        # minimal 3.10
python3 -m venv venv
source venv/bin/activate    # Windows: venv|Scripts|activate
pip install grpcio grpcio-tools
```
Praktikum dapat dijalankan pada satu komputer (dua terminal) atau dua komputer dalam satu jaringan. Untuk dua komputer, ganti localhost dengan alamat IP server dan pastikan port tidak diblokir firewall.

Ringkasan Konsep RPC
RPC membuat pemanggilan fungsi di mesin lain terlihat seperti pemanggilan fungsi lokal; stub di kedua sisi menyembunyikan detail jaringan.
Urutan satu pemanggilan RPC:
1. Client memanggil fungsi pada **client stub** seolah-olah fungsi lokal.
2. Client stub melakukan **marshalling**: mengemas nama prosedur dan parameter ke format pesan (XML, JSON, atau biner seperti Protobuf).
3. Pesan dikirim melalui jaringan (HTTP, HTTP/2, atau TCP).
4. **Server stub** (skeleton) menerima pesan dan melakukan **unmarshalling**.
5. Server stub memanggil prosedur sebenarnya di server.
6. Hasil dikemas kembali, dikirim ke client, lalu dibongkar oleh client stub dan dikembalikan ke pemanggil.

Stub menyembunyikan jaringan dari program client dan server



Permintaan: 1 panggil fungsi lokal · 2 marshalling lalu kirim pesan · 3 unmarshalling lalu eksekusi
Balasan: 4 kembalikan hasil · 5 marshalling hasil lalu kirim · 6 unmarshalling lalu return ke pemanggil

Alur satu pemanggilan RPC

Program client dan prosedur server tidak pernah menyentuh jaringan secara langsung; seluruh pengemasan dan pengiriman dikerjakan oleh kedua stub.

Istilah penting

Istilah	Arti singkat
Stub / skeleton	Kode perantara di sisi client / server yang menangani pengemasan dan pengiriman pesan
Marshalling	Mengubah struktur data menjadi aliran byte yang dapat dikirim
IDL	Interface Definition Language; kontrak antarmuka yang netral bahasa (contoh: file .proto)
Binding	Cara client menemukan server (alamat tetap, registry, atau service discovery)
Synchronous RPC	Client menunggu (blocking) sampai hasil diterima
Asynchronous RPC	Client melanjutkan pekerjaan dan menerima hasil kemudian (future/callback)
Idempoten	Operasi yang aman diulang karena hasilnya sama, misalnya get_saldo()

Semantik pemanggilan saat terjadi kegagalan

Semantik	Perilaku	Konsekuensi
Maybe	Tidak ada pengulangan	Prosedur mungkin tidak pernah dijalankan
At-least-once	Client mengulang sampai ada balasan	Prosedur bisa dijalankan lebih dari sekali; aman hanya untuk operasi idempoten
At-most-once	Server menyaring permintaan duplikat (ID permintaan)	Tidak pernah dijalankan dua kali, tetapi mungkin tidak dijalankan
Exactly-once	Ideal; sulit dicapai di jaringan nyata	Butuh penyimpanan persisten dan protokol tambahan

Perbedaan dengan panggilan lokal

- Tidak ada memori bersama: pointer dan referensi tidak bisa dikirim apa adanya.
- Latensi jauh lebih besar (mikrodetik vs milidetik).
- Kegagalan parsial: server, client, atau jaringan dapat gagal secara terpisah.
- Perbedaan representasi data antarmesin (endianness, encoding) harus ditangani oleh format serialisasi.

Latihan Soal Teori

Kerjakan sebelum Praktikum 1. Bagian A berisi 10 soal pilihan ganda, bagian B berisi 5 soal esai.

A. Pilihan ganda

1. Komponen yang bertugas mengemas parameter menjadi pesan di sisi client disebut…
   a. Skeleton b. Client stub c. Binder d. Dispatcher
2. Proses mengubah struktur data menjadi aliran byte untuk dikirim lewat jaringan disebut…
   a. Binding b. Routing c. Marshalling d. Multiplexing
3. Semantik yang menjamin prosedur tidak pernah dieksekusi lebih dari sekali adalah…
   a. Maybe b. At-least-once c. At-most-once d. Best-effort
4. Operasi berikut yang paling aman diulang (retry) secara otomatis adalah…
   a. transfer(a, b, 100) b. tambah_stok(id, 5) c. get_saldo(id) d. kirim_email(x)
5. Format serialisasi yang digunakan gRPC secara bawaan adalah…
   a. XML b. JSON c. YAML d. Protocol Buffers
6. Protokol transport yang dipakai gRPC adalah…
   a. HTTP/1.0 b. HTTP/2 c. FTP d. SMTP
7. Mengapa pointer tidak dapat dikirim langsung sebagai parameter RPC?
   a. Ukurannya terlalu besar b. Ruang alamat client dan server berbeda c. Dilarang oleh HTTP d. Pointer selalu bernilai null
8. Peran file .proto dalam gRPC adalah sebagai…
   a. File konfigurasi server b. IDL / kontrak layanan c. Log permintaan d. Sertifikat TLS
9. Pada RPC sinkron, apa yang dilakukan client setelah mengirim permintaan?
   a. Langsung menjalankan instruksi berikutnya b. Menunggu sampai balasan diterima c. Menutup koneksi d. Mengirim ulang setiap 1 ms
10. Tipe gRPC yang memungkinkan server mengirim banyak respons untuk satu permintaan adalah…
    a. Unary b. Server streaming c. Client streaming d. Broadcast

B. Esai

1. Jelaskan enam langkah yang terjadi dari saat client memanggil add(2, 3) sampai hasil 5 diterima, menggunakan istilah stub dan marshalling.
2. Sebutkan tiga perbedaan mendasar antara panggilan prosedur lokal dan RPC, serta dampaknya bagi programmer.
3. Sebuah layanan bank memakai semantik at-least-once untuk fungsi transfer(). Jelaskan masalah yang dapat muncul dan usulkan dua cara untuk mengatasinya.
4. Bandingkan XML-RPC, REST/JSON, dan gRPC dari sisi format data, kinerja, dan dukungan streaming.
5. Jelaskan perbedaan RPC sinkron dan asinkron, lalu berikan satu contoh kasus yang lebih cocok menggunakan RPC asinkron.

Pedoman jawaban esai

- **No. 3:** permintaan ulang setelah balasan hilang dapat menyebabkan transfer ganda. Solusi: ID permintaan unik yang dicatat server (deduplikasi, menjadi at-most-once), atau desain operasi idempoten (misalnya set_status(transaksi_id, selesai)).
- **No. 4:** XML-RPC berbasis teks XML, verbose dan lambat; REST/JSON lebih ringan dan mudah dibaca; gRPC memakai Protobuf biner di atas HTTP/2, paling efisien dan mendukung streaming dua arah.
- **No. 5:** contoh asinkron: client mengirim permintaan render video atau perhitungan berat lalu tetap melayani pengguna, dan hasil diambil lewat callback atau future.

Praktikum 1: RPC Dasar dengan XML-RPC

Tujuan: membangun server kalkulator dan client yang memanggilnya dari jarak jauh, lalu mengamati pesan XML yang dikirim.

Langkah 1 — Server (server_kalkulator.py)

```
from xmlrpc.server import SimpleXMLRPCServer

def tambah(a, b):
    return a + b

def kali(a, b):
    return a * b

def bagi(a, b):
    if b == 0:
        raise ValueError("Pembagian dengan nol")
    return a / b

server = SimpleXMLRPCServer(("0.0.0.0", 8000), allow_none=True,
logRequests=True)
server.register_introspection_functions()
server.register_function(tambah, "tambah")
server.register_function(kali, "kali")
server.register_function(bagi, "bagi")
print("Server XML-RPC berjalan di port 8000...")
server.serve_forever()
```

Langkah 2 — Client (client_kalkulator.py)

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/", verbose=True)

print("Daftar method:", proxy.system.listMethods())
print("5 + 3 =", proxy.tambah(5, 3))
print("4 x 7 =", proxy.kali(4, 7))

try:
    print(proxy.bagi(10, 0))
except xmlrpc.client.Fault as e:
    print("Error dari server:", e.faultString)
```

Langkah 3 — Jalankan dan amati

1. Jalankan server di terminal pertama: python server_kalkulator.py.
2. Jalankan client di terminal kedua: python client_kalkulator.py.
3. Karena verbose=True, client mencetak pesan HTTP dan XML yang dikirim. Salin satu pesan <methodCall> dan <methodResponse> ke laporan.

Latihan Praktikum 1

- ☐ **L1.1** Tambahkan fungsi pangkat(a, b) dan akar(x) di server; uji dari client.
- ☐ **L1.2** Buat fungsi statistik(daftar) yang menerima list angka dan mengembalikan dictionary berisi min, max, rata2. Amati bagaimana list dan dictionary dimarshalling menjadi XML.
- ☐ **L1.3** Kirim parameter berupa objek datetime dan bilangan sangat besar (lebih dari 2^{31}). Catat apa yang terjadi dan jelaskan penyebabnya.
- ☐ **L1.4** Jalankan server dan client di dua komputer berbeda. Ukur waktu 100 kali pemanggilan tambah( ) menggunakan time.perf_counter( ) dan bandingkan dengan 100 kali pemanggilan fungsi lokal.
- ☐ **L1.5** Matikan server ketika client sedang berjalan. Exception apa yang muncul? Tambahkan penanganan error yang sesuai.

Pertanyaan analisis: berapa ukuran (byte) pesan XML untuk tambah(5, 3)? Berapa persen yang merupakan data sebenarnya dan berapa persen overhead format?

Praktikum 2: gRPC dan Protocol Buffers

Tujuan: mendefinisikan layanan dengan IDL, menghasilkan stub secara otomatis, dan memakai dua pola pemanggilan: unary dan server streaming.

Langkah 1 — Kontrak layanan (akademik.proto)

```
syntax = "proto3";
package akademik;

service LayananNilai {
  rpc GetMahasiswa (MahasiswaRequest) returns (Mahasiswa);       // unary
  rpc TambahNilai  (NilaiRequest)     returns (StatusResponse);  // unary
  rpc StreamNilai  (MahasiswaRequest) returns (stream Nilai);    // server streaming
}

message MahasiswaRequest { string nim = 1; }
message Mahasiswa        { string nim = 1; string nama = 2; double ipk = 3; }
message NilaiRequest     { string nim = 1; string matkul = 2; double nilai = 3; }
message Nilai            { string matkul = 1; double nilai = 2; }
message StatusResponse   { bool sukses = 1; string pesan = 2; }
```

Langkah 2 — Generate stub

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. akademik.proto
```

Perintah ini menghasilkan akademik_pb2.py (kelas message) dan akademik_pb2_grpc.py (client stub dan kelas dasar server). Buka kedua file dan identifikasi kelas LayananNilaiStub serta LayananNilaiServicer.

Langkah 3 — Server (server_grpc.py)

```
import time
from concurrent import futures
import grpc
import akademik_pb2 as pb
import akademik_pb2_grpc as pb_grpc

DATA = {
   "2023001": {"nama": "Andi", "nilai": {"Basis Data": 85.0, "Jaringan": 78.0}},
   "2023002": {"nama": "Sari", "nilai": {"Basis Data": 92.0}},
}

class LayananNilai(pb_grpc.LayananNilaiServicer):
   def GetMahasiswa(self, request, context):
      mhs = DATA.get(request.nim)
      if mhs is None:
         context.abort(grpc.StatusCode.NOT_FOUND, f"NIM {request.nim} tidak
ditemukan")
      daftar = list(mhs["nilai"].values())
      ipk = sum(daftar) / len(daftar) / 25 if daftar else 0.0   # konversi sederhana 0–100
ke 0–4
      return pb.Mahasiswa(nim=request.nim, nama=mhs["nama"], ipk=round(ipk, 2))
```

```
    def TambahNilai(self, request, context):
        if not 0 <= request.nilai <= 100:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "Nilai harus 0–100")
        mhs = DATA.setdefault(request.nim, {"nama": "(baru)", "nilai": {}})
        mhs["nilai"][request.matkul] = request.nilai
        return pb.StatusResponse(sukses=True, pesan="Nilai tersimpan")

    def StreamNilai(self, request, context):
        mhs = DATA.get(request.nim)
        if mhs is None:
            context.abort(grpc.StatusCode.NOT_FOUND, "NIM tidak ditemukan")
        for matkul, nilai in mhs["nilai"].items():
            yield pb.Nilai(matkul=matkul, nilai=nilai)
            time.sleep(0.5)   # simulasi data dikirim bertahap

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    pb_grpc.add_LayananNilaiServicer_to_server(LayananNilai(), server)
    server.add_insecure_port("[::]:50051")
    server.start()
    print("Server gRPC berjalan di port 50051...")
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

Langkah 4 — Client (client_grpc.py)

```
import grpc
import akademik_pb2 as pb
import akademik_pb2_grpc as pb_grpc

with grpc.insecure_channel("localhost:50051") as channel:
    stub = pb_grpc.LayananNilaiStub(channel)

    mhs = stub.GetMahasiswa(pb.MahasiswaRequest(nim="2023001"))
    print(f"{mhs.nama} - IPK {mhs.ipk}")

    print(stub.TambahNilai(pb.NilaiRequest(nim="2023001", matkul="Komputasi
Paralel", nilai=88)).pesan)

    for n in stub.StreamNilai(pb.MahasiswaRequest(nim="2023001")):
        print(f"  {n.matkul}: {n.nilai}")

    try:
        stub.GetMahasiswa(pb.MahasiswaRequest(nim="9999"))
    except grpc.RpcError as e:
        print("Error:", e.code(), e.details())
```

Latihan Praktikum 2

- ☐ **L2.1** Tambahkan RPC client streaming KirimBanyakNilai(stream NilaiRequest) returns (StatusResponse) yang menerima banyak nilai sekaligus dan mengembalikan jumlah nilai yang tersimpan.
- ☐ **L2.2** Tambahkan RPC bidirectional streaming Diskusi(stream Pesan) returns (stream Pesan) untuk ruang diskusi sederhana antar-client.
- ☐ **L2.3** Tambahkan field baru string prodi = 4; pada message Mahasiswa. Jalankan client lama (stub belum digenerate ulang) terhadap server baru. Apakah tetap berjalan? Jelaskan konsep *backward compatibility* pada Protobuf.
- ☐ **L2.4** Bandingkan ukuran pesan: hitung len(mhs.SerializeToString()) dan bandingkan dengan pesan XML-RPC berisi data yang sama dari Praktikum 1.
- ☐ **L2.5** Ubah pemanggilan GetMahasiswa menjadi asinkron dengan stub.GetMahasiswa.future(...). Kirim 10 permintaan sekaligus, lalu kumpulkan hasilnya dengan .result().

Praktikum 3: Concurrency, Timeout, dan Toleransi Kegagalan

Tujuan: membuktikan bahwa retry tanpa pengaman dapat menjalankan operasi dua kali, lalu memperbaikinya dengan ID permintaan (semantik at-most-once) dan mengukur pengaruh jumlah thread server terhadap throughput.

Langkah 1 — Tambahkan RPC transfer ke akademik.proto

```
// di dalam service LayananNilai
rpc Transfer (TransferRequest) returns (StatusResponse);

message TransferRequest {
  string request_id = 1;
  string dari = 2;
  string ke = 3;
  double jumlah = 4;
}
```

Generate ulang stub setelah mengubah file .proto.

Langkah 2 — Server dengan deduplikasi dan respons lambat acak

```
import random, threading, time

SALDO = {"A": 1000.0, "B": 500.0}
PROSES = {}             # request_id -> StatusResponse yang sudah dikirim
LOCK = threading.Lock()

# tambahkan method ini ke kelas LayananNilai
def Transfer(self, request, context):
    with LOCK:
        if request.request_id in PROSES:        # permintaan duplikat
            return PROSES[request.request_id]
        if SALDO[request.dari] < request.jumlah:
            hasil = pb.StatusResponse(sukses=False, pesan="Saldo tidak cukup")
        else:
            SALDO[request.dari] -= request.jumlah
            SALDO[request.ke] += request.jumlah
            hasil = pb.StatusResponse(sukses=True, pesan=f"Saldo kini {SALDO}")
        PROSES[request.request_id] = hasil
    if random.random() < 0.3:
        time.sleep(3)     # simulasi balasan terlambat: transfer SUDAH terjadi
    return hasil
```

Langkah 3 — Client dengan timeout dan retry (exponential backoff)

```
import time, uuid
import grpc

def transfer_dengan_retry(stub, dari, ke, jumlah, maks=3):
    rid = str(uuid.uuid4())                 # ID sama untuk setiap percobaan
    for percobaan in range(1, maks + 1):
        try:
```

```
            req = pb.TransferRequest(request_id=rid, dari=dari, ke=ke, jumlah=jumlah)
            return stub.Transfer(req, timeout=2.0)
        except grpc.RpcError as e:
            bisa_ulang = e.code() in (grpc.StatusCode.DEADLINE_EXCEEDED,
                                      grpc.StatusCode.UNAVAILABLE)
            if not bisa_ulang or percobaan == maks:
                raise
            jeda = 0.5 * 2 ** (percobaan - 1)
            print(f"Percobaan {percobaan} gagal ({e.code().name}), ulang dalam {jeda} s")
            time.sleep(jeda)
```

Latihan Praktikum 3

- ☐ **L3.1** Jalankan 20 kali transfer 10 dari A ke B. Periksa saldo akhir: totalnya harus tetap 1500 dan A berkurang tepat 200.
- ☐ **L3.2** Ubah client agar membuat uuid baru di **setiap** percobaan. Ulangi L3.1, catat saldo akhir, dan jelaskan mengapa terjadi transfer ganda.
- ☐ **L3.3** Hapus with LOCK: lalu jalankan 50 transfer dari 10 thread client secara bersamaan (concurrent.futures.ThreadPoolExecutor). Apakah total saldo tetap 1500? Jelaskan race condition yang terjadi.
- ☐ **L3.4** Ukur throughput (permintaan per detik) untuk 200 pemanggilan GetMahasiswa dari 20 thread client, dengan time.sleep(0.1) ditambahkan di server, untuk max_workers = 1, 4, dan 10. Isi tabel di bawah dan buat grafiknya di laporan.
- ☐ **L3.5** (Pengayaan) Jalankan dua server di port 50051 dan 50052. Buat client yang otomatis berpindah ke server kedua jika server pertama UNAVAILABLE (failover).

max_workers	Waktu total (s)	Throughput (req/s)	Latensi rata-rata (ms)
1			
4			
10			

Pertanyaan analisis: server menyimpan PROSES di memori. Apa yang terjadi pada jaminan at-most-once jika server restart? Usulkan perbaikannya.

Tugas Proyek Mini: Sistem Terdistribusi Berbasis RPC

Setiap kelompok (2–3 orang) memilih **satu** studi kasus dan membangunnya dengan gRPC dalam waktu 1 minggu.

Pilihan studi kasus

No	Studi kasus	Fitur RPC wajib
1	**Reservasi tiket bioskop** beberapa client memesan kursi yang sama secara bersamaan	Unary PesanKursi, server streaming status kursi, penanganan race condition
2	**Komputasi terdistribusi (master–worker)** master membagi perhitungan bilangan prima 1–10.000.000 ke beberapa worker	Master memanggil 3+ worker secara paralel, menggabungkan hasil, mengukur speedup
3	**Chat room** banyak pengguna bertukar pesan secara real-time	Bidirectional streaming, daftar pengguna aktif, penanganan client terputus
4	**Key-value store terreplikasi** satu primary dan dua replica	Put/Get, replikasi ke replica via RPC, failover saat primary mati
5	**Antrian pengolahan gambar** client mengunggah gambar, server mengubahnya menjadi grayscale	Client streaming (unggah per chunk), status pekerjaan, RPC asinkron

Ketentuan umum

1. Antarmuka layanan didefinisikan dalam file .proto dengan minimal 4 RPC, termasuk minimal 1 RPC streaming.
2. Server menangani banyak client secara bersamaan tanpa data rusak (gunakan lock atau mekanisme lain dan jelaskan pilihannya).
3. Client menerapkan timeout dan retry; operasi yang tidak idempoten wajib dilindungi ID permintaan.
4. Demo dijalankan pada minimal 2 komputer atau 2 container berbeda.
5. Lakukan pengujian kinerja: latensi rata-rata dan throughput untuk 1, 5, dan 20 client bersamaan. Untuk studi kasus 2, hitung speedup = waktu 1 worker ÷ waktu N worker.
6. Kode disimpan di repositori Git dengan riwayat commit dari setiap anggota.

Rubrik penilaian

Aspek	Bobot	Sangat baik (85–100)	Cukup (60–84)	Kurang (< 60)
Desain antarmuka .proto	15%	Message dan RPC rapi, penamaan konsisten, memakai streaming dengan tepat	Berfungsi tetapi ada field atau RPC yang tidak perlu	Tidak memenuhi minimal 4 RPC
Fungsionalitas	25%	Semua fitur wajib berjalan di 2 mesin	Sebagian fitur berjalan	Program tidak berjalan
Concurrency & konsistensi data	20%	Tidak ada race condition, dibuktikan dengan uji beban	Ada lock tetapi tidak diuji	Data rusak saat diakses bersamaan
Toleransi kegagalan	15%	Timeout, retry, dan deduplikasi berjalan dan didemokan	Hanya timeout atau retry	Tidak ada penanganan kegagalan
Analisis kinerja	15%	Tabel dan grafik lengkap,	Data ada, analisis	Tidak ada

Aspek	Bobot	Sangat baik (85–100)	Cukup (60–84)	Kurang (< 60)
		analisis menjelaskan penyebab	dangkal	pengukuran
Laporan & presentasi	10%	Jelas, diagram arsitektur ada, semua anggota menguasai	Kurang lengkap	Tidak ada laporan

Praktikum 1–3 dinilai terpisah: setiap latihan (L1.x, L2.x, L3.x) bernilai sama, dan pertanyaan analisis menambah nilai laporan praktikum.

Format Laporan dan Pengumpulan

Laporan praktikum dikumpulkan paling lambat 1 minggu setelah pertemuan; tugas proyek dikumpulkan sebelum sesi presentasi.

Isi laporan praktikum

1. Identitas (nama, NIM, kelas, nomor praktikum).
2. Tujuan dan dasar teori singkat (maksimal 1 halaman).
3. Langkah kerja dan kode program setiap latihan.
4. Tangkapan layar hasil eksekusi server dan client.
5. Jawaban pertanyaan analisis dan hasil pengukuran (tabel dan grafik).
6. Kesimpulan dan kendala yang dihadapi.

Isi laporan proyek

1. Deskripsi studi kasus dan diagram arsitektur (komponen dan alur pemanggilan RPC).
2. Isi file .proto beserta penjelasan setiap RPC.
3. Strategi concurrency dan toleransi kegagalan yang dipakai.
4. Hasil pengujian kinerja dan analisisnya.
5. Pembagian tugas anggota kelompok.
6. Tautan repositori Git dan video demo (maksimal 5 menit).

Pengumpulan

- Format berkas: NIM_Praktikum-RPC-<nomor>.pdf untuk laporan praktikum dan Kelompok<no>_ProyekRPC.zip untuk proyek (berisi laporan PDF dan kode sumber).
- Kode harus bisa dijalankan dengan instruksi di README.md (cara instalasi, generate stub, dan menjalankan server/client).
- Kesamaan kode antarkelompok lebih dari batas wajar akan diperiksa; penggunaan referensi atau alat bantu wajib dicantumkan di laporan.